Event annotation reference — CNeuroMod videogame datasets

Every event type emitted into `sub-*/ses-*/func/*_desc-annotated_events.tsv`, what it means, and how it is read from the game's RAM.

Generated by `python -m videogames_utils.events.reference` from each dataset's `code/game_specific/` and annotated files; do not edit by hand. The machine-readable version of each vocabulary is the dataset's `task-<task>_events.json`, and each dataset's `code/annotations/README.md` carries its own part of this document with the row count of every event.

Dataset	Game	Frame rate	Event rows	Distinct `trial_type`	Repetitions
`mario`	Super Mario Bros. (NES)	60.099827 Hz	599,061	73	3,374
`mariostars`	Super Mario All-Stars: Super Mario Bros. (SNES)	60.098812 Hz	226,633	73	1,232
`mario3`	Super Mario Bros. 3 (NES)	60.099827 Hz	336,145	222	4,432
`shinobi`	Shinobi III: Return of the Ninja Master (Genesis)	59.922743 Hz	246,266	26	827

How to read this document

Event names are hierarchical: the part before the first `/` is the family, the parts after it are qualifiers. A `*` stands for a qualifier filled in at generation time with a decoded object name, e.g. `EnemyOnScreen/*` for `EnemyOnScreen/Goomba`.

Every segment is UpperCamelCase (`PlayerState/HitRecovery`).

Columns

Column	Meaning
`trial_type`	The event, from the game's controlled vocabulary.
`onset`	Seconds from the start of the fMRI run.
`duration`	Seconds; 0 for point events.
`frame_start`, `frame_stop`	Emulator frames, relative to the repetition.
`button`	Raw controller button behind an `Action/*` event; `n/a` otherwise.
`stim_file`	Path to the `.bk2`. **Set on the `Repetition` row only** — every other event sits inside exactly one of those windows. A repetition whose `.bk2` was lost keeps its row, with `stim_file` = `Missing file`, the onset (and, where the task logged it, the duration) from the plain `_events.tsv`, `n/a` frames and no events inside it.

Per-repetition metadata — `phase`, `IndexInRun`, `IndexGlobal`, `IndexLevel` and `Outcome` — is **not** repeated on event rows. It lives in the repetition's `gamelogs/*_summary.json`, keyed by the `stim_file` on the container row.

Onsets are computed at the console's true frame rate, read from the emulator core rather than assumed to be 60 (see the table above).

The container row

`Repetition` is one repetition of gameplay — one `.bk2` file. It is taken from the `gym-retro_game` row of the run's plain `_events.tsv` and is the only row that carries `stim_file`; every other event falls inside its window. `duration` is the replay's frame count divided by the true frame rate. It sorts first among rows sharing its onset, so slicing between container rows assigns every event to the right repetition.

Screens: what the game shows

The `Screen/*` rows say what the game is showing, as opposed to what the player is doing. They **partition every repetition**: the rows of a repetition are contiguous, do not overlap, and run from its first frame to its last, so exactly one is active at any frame and `Screen/Gameplay` is the complement of the rest. All are durational.

The `PlayerState` **form rows** tile the frames on which the player is *alive in the level*: every frame not inside a `Screen/Death`, `Screen/LevelIntro`, `Screen/GameOver` or `Screen/Map` row. Together, the two families give a form for every frame of play and a screen for every frame of the recording.

On-screen visibility

`EnemyOnScreen` and friends are **screen-visibility** events: they start when the object becomes visible and their `duration` runs until it leaves the screen or is defeated. An object that leaves and returns produces two events. The distinction being drawn is **on screen**, not **loaded in RAM**: slot occupancy alone means only that the engine is simulating the object, and games keep objects live well outside the camera. Each game's predicate and its validation are given in its section below.

Tracks shorter than 5 frames are dropped as slot churn, and gaps of up to 4 frames are bridged, because a one-frame flicker of the slot bookkeeping is not a disappearance the player perceives.

Controller actions

`Action/*` rows are normalised across games so that the same button role carries the same label, with the raw button preserved in the **`button`** column. `duration` spans the hold. `Action/Other` covers buttons with no documented function in the game. Each game's button mapping is given in its section below.

Repetition outcomes

Not events, but recorded per repetition in `gamelogs/*_summary.json` under `Outcome`:

Label	Meaning
`cleared`	The level was finished.
`failed/killed`	The final death was caused by contact.
`failed/fall`	The final death was a fall off the screen.
`failed/timeout`	The timer ran out.
`incomplete/warp`	The player left through a warp zone.
`incomplete/interrupted`	The recording stopped mid-level.

How each game decides it is given in its section below.

Validation

Independent checks on the generated events: the schema and controlled vocabulary (V0), invariants recomputed from the raw RAM by a different route than the generator (V1), and, for two datasets of the same game, agreement of the enemies met on each level (V2).

What none of that establishes is agreement with what is actually on screen. That needs the video review in `videogames_utils.events.review`, which samples events, cuts labelled clips from the frame-aligned recordings, and reports precision and recall per event type. Until it has been run, the accuracy figures quoted here are about internal consistency, not about ground truth.

Events per dataset

Rows of each event in each dataset; a dash where the game has no such event.

Event	mario	mariostars	mario3	shinobi
Action/Attack	—	—	—	53,626
Action/Down	3,244	773	4,472	23,466
Action/Jump	161,261	58,304	89,112	47,866
Action/Left	58,615	20,032	44,638	12,962
Action/Ninjutsu	—	—	—	19
Action/Other	0	16,489	0	0
Action/Right	110,648	37,082	61,681	53,535
Action/Run	26,205	11,383	15,817	—
Action/Select	0	0	0	0
Action/Start	0	0	0	0
Action/Up	891	211	5,960	6,550
AutoScrollStarted	—	—	962	—
BlockSmashed	5,450	2,024	842	—
CastleVisible	2,020	873	—	—
CheckpointReached	0	0	—	—
EnemyAttack/*	0	0	0	—
EnemyDefeated	—	—	—	19,632
EnemyDefeated/Projectile/*	6,519	3,277	2,255	—
EnemyDefeated/Shell/*	509	205	0	—
EnemyDefeated/Stomp/*	6,953	2,519	1,595	—
EnemyOnScreen/*	117,354	36,187	38,694	—
FlagpoleVisible	2,131	873	—	—
GoalCardCollected/*	—	—	253	—
GoalCardVisible	—	—	260	—
HealthGained	—	—	—	696
ItemCollected/*	—	—	—	3,170
ItemCollected/Coin	38,507	15,017	8,852	—
ItemCollected/Powerup	0	0	0	—
ItemOnScreen/*	5,679	1,963	2,571	—
LevelCompleted	2,053	883	253	520
LevelExited/Warp	148	84	—	—
LevelRestarted	0	0	2,474	—
LevelStarted	3,374	1,232	1,589	666
LifeGained	358	242	61	131
PSwitchExpired	—	—	61	—
PSwitchStarted	—	—	61	—
PipeEntered	1,013	350	0	—
PlayerDamaged	1,036	341	1,077	2,221

Event	mario	mariostars	mario3	shinobi
PlayerDied/Enemy	3,400	923	2,735	18
PlayerDied/Fall	1,762	498	942	126
PlayerDied/Timeout	0	0	19	—
PlayerState/Fire	276	133	27	—
PlayerState/Flying	—	—	660	—
PlayerState/Frog	—	—	1	—
PlayerState/Hammer	—	—	0	—
PlayerState/HitRecovery	1,036	545	1,033	2,221
PlayerState/KuriboShoe	—	—	6	—
PlayerState/Normal	—	—	—	773
PlayerState/Raccoon	—	—	184	—
PlayerState/Small	8,399	2,802	4,922	—
PlayerState/Star	440	205	153	—
PlayerState/Statue	—	—	3	—
PlayerState/Super	2,230	861	1,406	—
PlayerState/Tanooki	—	—	5	—
ProjectileAppeared/Shuriken	—	—	—	14,665
ProjectileOnScreen/*	0	0	21,367	—
Repetition	3,374	1,232	4,432	827
Screen/Death	5,162	1,421	3,696	144
Screen/GameOver	—	—	148	9
Screen/Gameplay	8,362	2,735	5,951	1,247
Screen/LevelEnd	2,053	883	253	520
Screen/LevelIntro	7,220	2,391	—	—
Screen/Map	—	—	1	—
Screen/Transition	1,147	428	1,889	648
ShellStartedMoving	0	0	2,772	—
TimerWarningStarted	232	1,232	0	—
WeaponPowerupExpired/*	—	—	—	0
WeaponPowerupStarted/*	—	—	—	8

Super Mario Bros. (NES) — `mario`; Super Mario All-Stars: Super Mario Bros. (SNES) — `mariostars`

Super Mario All-Stars reuses SMB1's object id table verbatim (verified empirically) and the same engine enumeration, so the `mario` (NES) and `mariostars` (SNES) datasets share this reference. Where the two ports differ, both variable names are given as *NES / SNES*.

The engine state variable

Most Super Mario Bros. events are read from one variable, the game's own statement of what the player is doing. It is `GameEngineSubroutine` at `$000E`, exposed as `player_state` on the NES and `player_action_state` on the SNES (Super Mario All-Stars reuses the same enumeration). Values, from the disassembly's dispatch table:

value	routine	used for
0	`Entrance_GameTimerSetup`	`Screen/LevelIntro` (the black title card, at a level start or after a life loss) or `Screen/Transition` (the ~24-frame black screen between areas)
1	`Vine_AutoClimb`	`Screen/Transition`
2	`SideExitPipeEntry`	`PipeEntered`, `Screen/Transition`
3	`VerticalPipeEntry`	`PipeEntered`, `Screen/Transition`
4	`FlagpoleSlide`	`LevelCompleted` (fallback), start of `Screen/LevelEnd`
5	`PlayerEndLevel`	`LevelCompleted`, `Screen/LevelEnd`
6	`PlayerLoseLife`	`PlayerDied/Fall` (end of; see the fall-timing note), end of `Screen/Death`
7	`PlayerEntrance`	`Screen/Transition` (Mario emerging from a pipe or dropping into the area)
8	`PlayerCtrlRoutine`	normal play, `Screen/Gameplay`
9	`PlayerChangeSize`	start of `PlayerState/Super` (the status byte changes on this frame)
10	`PlayerInjuryBlink`	`PlayerDamaged`; the form row ends and `PlayerState/HitRecovery` starts
11	`PlayerDeath`	`PlayerDied/Enemy`, start of `Screen/Death`
12	`PlayerFireFlower`	start of `PlayerState/Fire`

Using the engine rather than counters matters because counters lag: `lives` only drops once the death animation has finished, several seconds after the death itself.

On-screen visibility

Dataset	Predicate	Validation
mario	`Enemy_Flag` set **and** `EnemyOffscrBitsMasked == 0`	horizontally inside the visible band on 97.5–98.9% of frames when 0, 0.0–1.5% when non-zero; `enemy_y` inside 0–239 on 99.9% of accepted frames, so no separate vertical test is needed
mariostars	bit 0 of `sprite_onscreen_flag_*`	no per-sprite coordinates exist in this port, so the predicate is checked indirectly by cross-port agreement with the NES on the 21 shared levels

Scenery is the one deliberate exception and uses slot occupancy on its own, meaning "spawned" rather than "on screen": `FlagpoleVisible` / `CastleVisible` (scenery never clears the enemy offscreen test — 0 of 479 frames in a sampled replay), and any repetition whose `_variables.json` predates the visibility addresses.

Screens

Row	Meaning	Detection
`Screen/Gameplay`	The player controls the character in the level.	engine 8, 9, 10, 12
`Screen/LevelIntro`	The level's title card.	engine 0 (NES) / `OperMode_Task` ≠ 3 (SNES) at the start of the repetition or directly after a death: the black "WORLD x-y / Mario × n" card, 86-193 frames
`Screen/Death`	The death sequence, no control.	engine 11 through 6, or from the fall (see below) to 6
`Screen/LevelEnd`	The end-of-level sequence, no control.	engine 4 then 5: flagpole slide, walk into the castle, time bonus, 6-14 s
`Screen/Transition`	Between areas, no control.	engine 1, 2, 3, 7 and the 0 / task ≠ 3 stretch between them: pipe travel, vine climb, the exit from a bonus area, 24 frames of black

Every boundary was checked on rendered emulator frames. The SNES title card needs `OperMode_Task` ($0772), which the engine variable cannot see (it idles at 8 while the card is up); it is read out of the shipped `reset` variable, a 4-byte little-endian BCD read at $0771 whose second byte pair is that task (1 =

card, 3 = game routines). The SNES fades in and out around the card, so those boundaries are within ~30 frames (0.5 s).

Timing a fall death

`PlayerLoseLife` does not run when the player drops into a pit — it runs about **4 s later**, once the game has let the player fall out of the level and is ready to reset. Recording the death there puts it seconds after anything the player saw, so `onset` is walked back to the frame the player actually left the screen and `duration` runs from there to the life loss. The old timing is still available as `onset + duration`.

Dataset	How the fall is dated	Accuracy
mario	`Player_Y_HighPos` (`player_y_screen`) leaving 1 — it is 1 on 95.0% of play frames, 0 above the screen, >1 below it	exact; 69 of 69 sampled fall deaths have the crossing, at a lag of 244–257 frames
mariostars	the frame the level timer stops ticking — the game freezes gameplay the moment the player drops out — plus half a tick	validated by running the same proxy on the NES against the exact crossing: median error −12 frames, **100% within 24 frames (0.4 s)**; the half-tick correction removes that bias

The SNES port exposes no player-Y address, which is why it needs the proxy. The `scroll_x_low` freeze was also tested as a proxy and is worse (only 77% within 0.4 s, p10 −50 frames), so the timer is used.

Player

Event	Description	Detection
`PlayerDamaged`	Hit, losing the current power-up state.	`player_state` / `player_action_state` transitions to **10** (`PlayerInjuryBlink`).
`PlayerDied/Enemy`	Killed by an enemy or other damaging object.	Engine transitions to **11** (`PlayerDeath`).
`PlayerDied/Fall`	Fell into a pit.	Engine transitions to **6** (`PlayerLoseLife`) with no **11** in the preceding 400 frames — a fall bypasses the death routine. `onset` is walked back to the frame the player left the screen (see below); `duration` runs to the life loss.
`PlayerDied/Timeout`	The level timer ran out.	Engine reaches **11** while `time` / `time_units` has been 0 for the preceding 120 frames.
`LifeGained`	Gained an extra life.	`lives` increases.
`PlayerState/Small`, `PlayerState/Super`, `PlayerState/Fire`	The form the player is in, one row per continuous stretch. Together they tile the frames on which the player is alive in the level.	Maximal runs of `player_status` == **0** / **1** / **2** over the alive frames (outside `Screen/Death` and `Screen/LevelIntro`). `PlayerStatus` is written on the very frame the item is collected (engine **9** / **12**) and on the hit frame (engine **10**), so the row starts and ends on those frames. A row is cut at a death and Small resumes once the title card has gone.
`PlayerState/Star`	Star invincibility, whole period.	Non-zero run of `star_timer` (`StarInvincibleTimer`, $079F / $07AF): 35 → 0 on the 21-frame interval clock, ~730 frames.
`PlayerState/HitRecovery`	Post-hit invulnerability: the ~3.7 s after a hit during which `InjurePlayer` ignores enemy contact and the player blinks.	Non-zero run of `injury_timer` (`InjuryTimer`, $079E / $07AE, **new**): 8 → 0 on the interval clock, paused during the shrink animation.

> On the SNES, `player_status`, `injury_timer` and `star_timer` are new in this release ($7E0756, $7E07AE, $7E07AF). The shipped `player_powerup` ($0578) and `star_power_timer` ($0553) hold unrelated bytes

> (values like 82 and 107, constants rather than a countdown) and are no longer read; see `ram.SMAS_CANDIDATES` for how the real ones were found.
>
> `player_status` is a variable added in this release, reading `PlayerStatus` at `$0756` as a single byte. The pre-existing `powerstate` reads the same address as 4-byte BCD, which mixes in the neighbouring death flag and reports values like 10000. Use `player_status`.

Items and blocks

Event	Description	Detection
`ItemCollected/Coin`	Collected a coin.	`coins` / `player_coins` increases, or drops by ≥99 (the counter wraps at 100 and awards a life).
`ItemOnScreen/Powerup`	A mushroom, flower or star became visible.	An enemy slot holds object id **`0x2E`** (`PowerUpObject`) and is visible.
`BlockSmashed`	Destroyed a brick from below.	`score` increases by exactly **5** (NES, BCD in units of 10 points) or **50** (SNES, actual points) while the player is airborne (`jump_airborne` ∈ {1,2} on the NES), before the level ends. The end-of-level time bonus produces the same increment, so scoring stops at the level end.

Enemies

Event	Description	Detection
`EnemyOnScreen/{type}`	An enemy became visible.	A visible track opens in an object slot. Type from `enemy_id_*` / `sprite_number_*` decoded through the SMB1 object table. `duration` spans the visible lifetime.
`EnemyDefeated/Stomp/{type}`	Defeated by jumping on it.	`Enemy_State` / `sprite_state_*` transitions into **4** from a non-kill state.
`EnemyDefeated/Projectile/{type}`	Defeated by a fireball or similar.	The same array transitions into **34**.
`EnemyDefeated/Shell/{type}`	Defeated by a moving shell.	The same array transitions into **132**.
`ProjectileOnScreen/{type}`	A Bowser flame is on screen.	A visible track whose object id is **`0x15`** (BowserFlame). Bullet Bills are **not** here — see below.

The defeat is emitted at the **first** frame of the kill state, i.e. when the enemy is hit — not when its corpse finishes falling. This matters because on a projectile kill the game deliberately moves the dying enemy's bounding box off screen, so reading the end of the animation would place the event after the enemy stopped being visible.

A defeat names both the method and the species, e.g. `EnemyDefeated/Stomp/Goomba`.

Object ids that are not enemies and are excluded from `Enemy_*`: moving platforms and lifts (`0x24`–`0x2C`), items and scenery (`0x16`, `0x2E`, `0x2F`, `0x30`, `0x31`, `0x32`, `0x35`, `0x36`), spawner/control objects (`0x17`, `0x18`) and the Bowser flame (`0x15`).

Bullet Bills are enemies, and live under two ids

A Bullet Bill can be stomped for points and kills the player on contact, so it is an enemy rather than a projectile. It occupies **two** object ids depending on which spawner made it — `0x08` from a frenzy, `0x33` from a cannon — and the disassembly names both after the spawner rather than after the object. Both now decode to `BulletBill`.

`0x33` used to be classed as a spawner and discarded outright, which threw away most of the Bullet Bills in both datasets: over 120 mario replays it accounts for **331** flying runs against 90 for `0x08` (mariostars: 283

against 43). Runs of the two ids are indistinguishable in motion — both travel at a median **1.5 px/frame**. On one sampled session this took `BulletBill` from 25 tracks to 267.

The id is shared with the stationary cannon that fires them. On the NES, where per-slot X is available, a track is kept only if it moves in X on at least half its frames; 90% of visible tracks pass. The SNES port exposes no per-sprite X, so the test cannot run there and roughly 10% of its `EnemyOnScreen/BulletBill` tracks are expected to be the cannon.

Piranha Plants count only while out of the pipe

A Piranha Plant holds its slot for the whole cycle — rising, waiting, retracting, then sitting inside the pipe — and the offscreen bits describe only the horizontal edges, so the slot reads as visible even while the plant is hidden and cannot touch the player.

Its screen Y traces the cycle exactly: over 60 mario replays the plant moves through 25 distinct Y values spanning **24 px**, resting at the maximum. Within each run of the slot the resting position is that maximum, and the plant is out whenever it sits above it. A run that never shows the full 24 px cannot identify its own resting position and is left visible throughout rather than guessed at.

34.2% of what used to count as visible Piranha Plant time was the plant hidden in its pipe. One long track per pipe becomes one event per emergence: on a sampled session, 141 tracks of median 3.06 s became 228 of median 1.72 s.

`Enemy_State` was checked first and is **0** for the entire cycle, so it cannot serve — which is why the SNES port, exposing no per-sprite Y, keeps the old behaviour.

Note that **id `0x00` is Green Koopa**, a real enemy — a zero id does not mean an empty slot. Slot occupancy is given by `Enemy_Flag`, not by the id.

Environment and level

Event	Description	Detection
`PipeEntered`	Entered a pipe.	Engine transitions to **2** (`SideExitPipeEntry`) or **3** (`VerticalPipeEntry`).
`FlagpoleVisible`	The end-of-level flagpole came on screen.	An object slot holds id **`0x30`** (`FlagpoleFlagObject`). Judged by slot occupancy alone, because scenery never clears the enemy offscreen test.
`CastleVisible`	The end-of-level star flag / castle came on screen.	The same, with id **`0x31`** (`StarFlagObject`).
`TimerWarningStarted`	The game began warning that time is short.	NES: bit **`0x40`** of `event_music_queue` (the game's own "time running out" music cue). SNES: the timer crossing 100 downward during normal play.
`LevelStarted`	A gameplay attempt began.	Frame 0 of the repetition.
`LevelCompleted`	The level was finished.	Engine transitions to **5** (`PlayerEndLevel`), falling back to **4** (`FlagpoleSlide`). Using `PlayerEndLevel` means this also fires on castle levels, which end at an axe rather than a flagpole.
`LevelExited/Warp`	The player left through a warp-zone pipe.	The `world` / `area` index changes **and** the level never ended. Only fires on w1-2 and w4-2, the two warp-zone levels.

> **`CheckpointReached` is deliberately not emitted.** Its only candidate signal, `HalfwayPage` (`$075B`), is written when the player *dies* past the midpoint in order to place the respawn — not when the midpoint is crossed. In a sampled w1-1 replay it is non-zero only during the two death/respawn sequences.

Controller actions

Event	NES	SNES
`Action/Left`, `Action/Right`, `Action/Up`, `Action/Down`	D-pad	D-pad
`Action/Jump`	`A`	`B`, `A`
`Action/Run`	`B`	`Y`, `X`
`Action/Start`	`START`	`START`
`Action/Select`	`SELECT`	`SELECT`
`Action/Other`	—	`L`, `R`

The SNES `L` and `R` buttons have no documented function in this game; the previous pipeline dropped them entirely (`L` is pressed 30 times in a single sampled replay).

Repetition outcomes

The label describes how the repetition *ended*, so when several lives are lost only the final death is classified. The previous logic tested `jump_airborne == 3`, which is the flagpole slide **and** the vine climb, so warps taken up a vine were labelled `cleared`; 71 mario and 59 mariostars repetitions changed label when this was corrected.

Provenance

RAM addresses and value tables are taken from published sources and each was verified against an independent signal before being used:

- **Super Mario Bros.** — 1wErt3r's SMBDIS.ASM
- **Super Mario All-Stars** — Data Crystal's partial RAM map, plus the empirical finding that its object ids match SMB1's verbatim

Cross-port agreement between the NES and SNES versions (V2): 15 of the 21 levels containing enemies yield identical rosters on the two consoles.

Super Mario Bros. 3 (NES) — `mario3`

Object ids come from `Level_ObjectID` (`$0671`, 8 slots), decoded through the 170-entry `OBJ_*` table generated from the disassembly. Verified: a World 1-1 replay decodes to Goomba, ParaGoomba, RedTroopa and VenusFireTrap — that level's actual roster.

On-screen visibility

Dataset	Predicate	Validation
mario3	`Objects_State != 0` **and** `Objects_SprHVis == 0` **and** `Objects_SprVVis == 0`	`SprHVis == 0` is horizontally inside the band on 99.9% of frames (n=9571) vs 11.7% when non-zero; among frames passing that test, `SprVVis == 0` is vertically inside the band on 100.0% (n=37436) vs 5.1% when non-zero

SMB3 is the game where the two notions diverge most: it scrolls vertically, so a slot can be horizontally on screen while sitting entirely above or below it. The vertical flag removes 21% of the frames the horizontal test alone would have called visible.

Screens

Row	Meaning	Detection
`Screen/Gameplay`	The player controls the character in the level.	everything not below

Row	Meaning	Detection
`Screen/Death`	The death sequence, no control.	`player_is_dying` ≠ 0, ~200 frames
`Screen/LevelEnd`	The end-of-level sequence, no control.	from the goal card to the end of the recording: card animation and COURSE CLEAR, ~340 frames
`Screen/Transition`	Between areas, no control.	`player_halt_game` ≥ 128 while the next area loads, plus the pipe-entry hold before it
`Screen/GameOver`	The GAME OVER screen.	`level_music` = 0 with `lives` < 0 after the death: the map with the GAME OVER dialog. **148 of 4063 recordings** ran on into this instead of stopping at the death, for a median 18 s and up to 4 min.
`Screen/Map`	The SMB3 world map.	`level_music` = 0 with `lives` ≥ 0 after the death: one recording (sub-06 ses-010 w6l4 rep-000) returned to the map and recorded a second attempt at the level

Timing a fall death

Dataset	How the fall is dated	Accuracy
mario3	unchanged: `player_is_dying == 2` already fires at the drop, 210 frames (3.5 s) *before* the life is lost	the game's own flag

Player

Event	Description	Detection
`PlayerDamaged`	Hit, losing the current suit.	`player_suit_lost` (the suit-lost poof counter) becomes non-zero, falling back to `invisibility_timer` (`Player_FlashInv`, the post-hit blink). Suppressed within 120 frames of a death.
`PlayerDied/Enemy`	Killed by an enemy.	`player_is_dying` (`$00F1`) becomes **1**.
`PlayerDied/Fall`	Dropped off the bottom of the screen.	`player_is_dying` becomes **2**.
`PlayerDied/Timeout`	Time ran out.	`player_is_dying` becomes **3**.
`LifeGained`	Gained an extra life.	`lives` increases.
`PlayerState/{Small,Super,Fire,Raccoon,Frog,Tanooki,Hammer}`	The suit the player wears, one row per continuous stretch. Together they tile the frames on which the player is alive in the level.	Maximal runs of a constant `powerup` (`Player_Suit`, `$00ED`) over the alive frames (outside `Screen/Death`, `Screen/GameOver`, `Screen/Map`). A hit as Fire or in a suit drops the player to Super, so one row ends and the next starts on the same frame.
`PlayerState/Star`	Star invincibility.	`invincibility_timer` (`Player_StarInv`) non-zero run, ~448 frames.

Event	Description	Detection
PlayerState/HitRecovery	Post-hit invulnerability while blinking, ~113 frames.	invisibility_timer (Player_FlashInv) non-zero run.
PlayerState/Flying	Flying with a flight-capable suit.	flight_timer (Player_FlyTime) non-zero run.
PlayerState/Statue, PlayerState/KuriboShoe	Tanooki statue form; riding Kuribo's Shoe.	statue_timer / kuribo_shoe non-zero run. Both are rare (1 and 5 repetitions in the corpus).
PSwitchStarted, PSwitchExpired	A P-Switch is active.	p_switch_timer (Level_PSwitchCnt) non-zero run.

> player_is_dying states the cause of death outright. It replaces the previous timer-freeze plus X-position heuristic, and moved 942 repetitions from the catch-all failed/killed to failed/fall.

Objects

Event	Description	Detection
EnemyOnScreen/{type}	An enemy became visible.	A visible track opens in an object slot (Objects_State != 0 and both Objects_SprHVis and Objects_SprVVis clear); type from object_id_*.
EnemyDefeated/Stomp/{type}	Stomped.	object_state_* enters **7** (OBJSTATE_SQUASHED), or enters **3** (OBJSTATE_SHELLED) from a non-shell state — a stomp that shells an enemy.
EnemyDefeated/Projectile/{type}	Killed by a projectile or tail.	object_state_* enters **6** (OBJSTATE_KILLED) or **8** (OBJSTATE_POOFDEATH).
ShellStartedMoving	A shelled enemy was kicked.	object_state_* enters **5** (OBJSTATE_KICKED).
ProjectileOnScreen/{type}	A hammer, fireball, cannonball etc. became visible.	special_obj_id_* (SpecialObj_ID) non-zero, decoded through the 22-entry SOBJ_* table.
ItemOnScreen/{1Up,Starman,Mushroom,FireFlower,SuperLeaf}	A power-up became visible.	Object ids 0x0B, 0x0C, 0x0D, 0x19, 0x1E.
ItemCollected/Coin	Collected a coin.	coins_p1 increases, or wraps at 100.
BlockSmashed	Destroyed a brick.	score increases by exactly **1** (10 points) while in_air is set.

Sixty object ids are excluded from Enemy_* as scenery, platforms, items or engine control objects. The exclusion list is derived from the disassembly's own OBJ_* names rather than hand-written, so it stays correct if the table is regenerated.

Level

Event	Description	Detection
`LevelStarted`	First attempt at this level in the run.	Frame 0 of a `rep-000` file.
`LevelRestarted`	A retry after dying.	Frame 0 of a `rep-001+` file. In SMB3 one level attempt spans up to three one-life `.bk2` files.
`GoalCardVisible`	The end-of-level roulette card is on screen, 0.5-4 s before it is taken. Untyped: its face cycles until touched.	A visible track of object id **`0x41`** (`OBJ_ENDLEVELCARD`). An earlier release keyed on the card items `0x21`-`0x23` (`PowerUpMushCard` etc.), which never appear at a level end, so the event never fired.
`LevelCompleted`	Finished the level.	A goal card enters the inventory (`goal_cards_p1_{1,2,3}` increases).
`GoalCardCollected/{Mushroom,Flower,Star,…}`	Which card was collected.	The same transition; the card value names it.
`AutoScrollStarted`	The level began scrolling on its own.	`level_hautoscroll` (`Level_HAutoScroll`) becomes non-zero.

Controller actions

Event	NES
`Action/Left`, `Action/Right`, `Action/Up`, `Action/Down`	D-pad
`Action/Jump`	`A`
`Action/Run`	`B`
`Action/Start`	`START`
`Action/Select`	`SELECT`

Repetition outcomes

The cause of death comes from `player_is_dying`, which dropped the catch-all `failed/killed` from 91% to 67% of repetitions.

Provenance

RAM addresses and value tables are taken from the published disassembly and each was verified against an independent signal before being used:

- **Super Mario Bros. 3** — captainsouthbird's disassembly

Shinobi III: Return of the Ninja Master (Genesis) — `shinobi`

This dataset's events are materially less well characterised than those of a game with a public RAM map, and should be read with that in mind. There is no public RAM map for Shinobi III, and roughly half the entries in its `data.json` are background/palette scratch (`backTree`, `backbrush`, …) with no gameplay meaning. Everything below is derived from a counter whose meaning is unambiguous.

On-screen visibility

The game has no object visibility at all: nothing in the available RAM map locates enemy objects, so `EnemyDefeated` is inferred from score deltas.

Screens

Row	Meaning	Detection
`Screen/Gameplay`	The player controls the character in the level.	`blackScreen` 0 or 40 (40 is the scroll lock of an enemy wave, not a black screen)
`Screen/Death`	The death sequence, no control.	from the frame `health` reaches 0 to the life loss: 127-344 frames of animation and fade
`Screen/LevelEnd`	The end-of-level sequence, no control.	the end-of-level fade running to the end of the recording (`blackScreen` 22 or 62): ~380 frames of ROUND CLEAR tally on level 1, 15-16 frames on levels 4 and 5
`Screen/Transition`	Between areas, no control.	section fades (22 or 62 then 20, ~80 frames) and the fade back in after a death (21 then 20, ~85 frames)
`Screen/GameOver`	The GAME OVER screen.	`blackScreen` 21 with `lives` < 0, 3 recordings

Events

Event	Description	Detection
`PlayerDamaged`	Hit; the health bar decreased.	`health` decreases, except for the drop to 0 that starts a death, which is the death rather than a hit.
`HealthGained`	Collected a health item.	`health` increases.
`PlayerDied/Enemy`	Health drained by enemy hits.	The player's animation state (`status`, $FF415A) enters **41**, on the frame `health` reaches 0: the player collapses in the play area, 1-3 health left before the last hit. A point event; the sequence that follows is the `Screen/Death` row. 18 deaths.
`PlayerDied/Fall`	Fell into a pit or into water. Durational.	`status` enters **43**, on the frame `health` reaches 0: the player is at the bottom edge of the screen and the health bar is wiped in one frame, usually from full. `duration` runs to the life loss -- `lives` only drops once the death animation and fade have played, 2-6 s later -- or to the end of the recording when it stops first (4 deaths). 126 deaths.
`LifeGained`	Gained a life.	`lives` increases.
`EnemyDefeated`	Defeated an enemy. **Untyped.**	`instantScore` increases by one of the documented enemy values: **200** (basic enemies), **300** (mortars, machineguns), **400** (cauldron-heads), **500** (anti-riot cop, hovering ninja).
`ProjectileAppeared/Shuriken`	Threw a shuriken.	`shurikens` decreases by exactly 1.
`ItemCollected/Shurikens`	Picked up shurikens.	`shurikens` increases. The amount is not carried on the row; it is recoverable from `shurikens` in the repetition's `_variables.json`.
`WeaponPowerupStarted/Ninjutsu{n}`	Used a ninjutsu.	`ninjitsu` decreases; `typeOfNinjitsu` names it. Happens in only 8 of 666 repetitions, and `typeOfNinjitsu` is always 0, so ninjutsu is not tracked as a state.
`PlayerState/HitRecovery`	The game's post-hit counter, during which the player flashes.	Non-zero run of `hit_timer` (**new**, $FF4165 -- the byte at raw offset $FF4164 of the word-swapped work RAM) that begins within 3 frames of a `PlayerDamaged`: 80 or 64 → 0, one per frame. Runs from 48 (knock-backs costing no health) and the 25-frame idles at 1 are ignored. Cut where `health` reaches 0.
`PlayerState/Normal`	Shinobi's single form; present so that every dataset has a form for every frame of play.	One row per stretch of frames on which the player is alive (outside `Screen/Death` and `Screen/GameOver`). The temporary weapon upgrade is not tracked.

Event	Description	Detection
`LevelStarted`	A gameplay attempt began.	Frame 0.
`LevelCompleted`	Finished the level.	The start of the end-of-level fade that closes the recording: the last `blackScreen` stretch of the repetition holds **22** or **62** (see Screens above). Over the corpus this is present in 519 of 532 cleared repetitions and in none of the 134 failed ones; the 13 misses are recordings that stopped inside the last enemy wave, before the fade.

Known limitations

- **No `EnemyOnScreen` / `Enemy_counter`.** Nothing in the available RAM map locates enemy objects.
- **`EnemyDefeated` is untyped**, and only the four documented score values are counted. Other increments occur — 1000 is common, and 250/350/700/3000/5000 appear — but nothing available attributes them, so they are deliberately not counted rather than guessed at. Roughly a third of scoring events are therefore not represented. (The previous pipeline counted only 200 and 300, silently dropping the 400- and 500-point kills its own docstring listed.)
- **`LevelCompleted` depends on the recording reaching the fade.** The previous pipeline fabricated one five seconds before the end of any repetition in which no life was lost — a property of the whole repetition rather than a moment in time. An earlier release of this module then dropped the event, having found `blackScreen` set in every repetition; it is, but that value (40) is the scroll lock of an enemy wave, and the *trailing* fade value discriminates perfectly (see the `LevelCompleted` row above). 13 cleared repetitions whose recording stopped inside the last enemy wave get no event; their `_summary.json` `Outcome` still says `cleared`.
- `PlayerDied/*` and `LifeGained` are **new**: the previous pipeline had no player-death event at all, despite `lives` recording it unambiguously. The cause comes from the death state of `status`, which takes no other value at any other time; checked on rendered frames for all 144 deaths. The game has no level timer, so there is no `PlayerDied/Timeout`, and no other death occurs in levels 1, 4 and 5.

Controller actions

Event	Genesis
`Action/Left`, `Action/Right`, `Action/Up`, `Action/Down`	D-pad
`Action/Jump`	`C`
`Action/Attack`	`B`
`Action/Ninjutsu`	`A`
`Action/Start`	`START`
`Action/Select`	`MODE`
`Action/Other`	`X`, `Y`, `Z`

Repetition outcomes

The outcome is "was a life lost", the only signal the RAM map supports.

Provenance

No public RAM map exists for Shinobi III; everything here is derived from counters whose meaning is unambiguous.